

Java Programming: From C

Chapter: 05. Syntax and Language Ergonomics

Prepared by: Chunghyun Lim (Matriculated in 2019)

Last Updated: June 2026



목차

1. foreach 문
2. var 키워드

1. foreach 문

foreach 문이란?

- foreach 문은 Java의 향상된 for 문(enhanced for statement)을 가리키는 표현이다.
- foreach 문은 배열이나 컬렉션에 저장된 데이터를 순차적으로 순회하기 위한 편리한 반복문이다.
- 배열이나 컬렉션의 내부 구조나 접근 방식을 직접 고려하지 않고 저장된 데이터 자체에 집중할 수 있도록 해준다.

```
for (자료형 변수명 : 배열 또는 컬렉션) {  
    }  
}
```

1. foreach 문

ArrayList에서의 foreach 문

```
ArrayList<String> userNames = new ArrayList<>();  
userNames.add("Alice");  
userNames.add("Bobby");  
userNames.add("Charlie");  
  
for (String userName : userNames) {  
    // userName을 활용한 로직  
}
```

```
{ } userNames = {ArrayList@854}  
{ } 0 = "Alice"  
{ } 1 = "Bobby"  
{ } 2 = "Charlie"  
{ } userName = "Alice"  
-----  
{ } userNames = {ArrayList@854}  
{ } 0 = "Alice"  
{ } 1 = "Bobby"  
{ } 2 = "Charlie"  
{ } userName = "Bobby"  
-----  
{ } userNames = {ArrayList@854}  
{ } 0 = "Alice"  
{ } 1 = "Bobby"  
{ } 2 = "Charlie"  
{ } userName = "Charlie"
```

- foreach 문은 ArrayList에 저장된 요소를 처음부터 마지막까지 순차적으로 순회한다.

1. foreach 문

HashSet에서의 foreach 문

```
HashSet<String> processedEmails = new HashSet<>();  
processedEmails.add("alice@example.com");  
processedEmails.add("bobby@example.com");  
processedEmails.add("charlie@example.com");  
  
for (String email : processedEmails) {  
    // email을 활용한 로직  
}
```

```
{ } processedEmails = {HashSet@854}  
{ } 0 = "charlie@example.com"  
{ } 1 = "alice@example.com"  
{ } 2 = "bobby@example.com"  
{ } email = "charlie@example.com"  
  
{ } processedEmails = {HashSet@854}  
{ } 0 = "charlie@example.com"  
{ } 1 = "alice@example.com"  
{ } 2 = "bobby@example.com"  
{ } email = "alice@example.com"  
  
{ } processedEmails = {HashSet@854}  
{ } 0 = "charlie@example.com"  
{ } 1 = "alice@example.com"  
{ } 2 = "bobby@example.com"  
{ } email = "bobby@example.com"
```

- foreach 문은 HashSet에 저장된 모든 요소를 순회하며 순회 순서는 보장되지 않는다.

1. foreach 문

HashMap에서의 foreach 문

```
HashMap<String, Long> lastLoginTimesByUserName = new HashMap<>();
lastLoginTimesByUserName.put("alice", 1735689600L);
lastLoginTimesByUserName.put("bobby", 1740830400L);
lastLoginTimesByUserName.put("charlie", 1750002600L);

for (Map.Entry<String, Long> entry : lastLoginTimesByUserName.entrySet()) {
    // entry.getKey(), entry.getValue()를 활용한 로직
}
```

- foreach 문은 HashMap에 저장된 모든 key-value 쌍을 순회하며 순회 순서는 보장되지 않는다.

```
{ } lastLoginTimesByUserName = {HashMap@854}
{ } "bobby" -> {Long@863} 1740830400
{ } "alice" -> {Long@865} 1735689600
{ } "charlie" -> {Long@867} 1750002600
{ } entry = {HashMap$Node@859}
{ } key = "bobby"
{ } value = {Long@863} 1740830400
```

```
{ } lastLoginTimesByUserName = {HashMap@854}
{ } "bobby" -> {Long@863} 1740830400
{ } "alice" -> {Long@865} 1735689600
{ } "charlie" -> {Long@867} 1750002600
{ } entry = {HashMap$Node@860}
{ } key = "alice"
{ } value = {Long@865} 1735689600
```

```
{ } lastLoginTimesByUserName = {HashMap@854}
{ } "bobby" -> {Long@863} 1740830400
{ } "alice" -> {Long@865} 1735689600
{ } "charlie" -> {Long@867} 1750002600
{ } entry = {HashMap$Node@861}
{ } key = "charlie"
{ } value = {Long@867} 1750002600
```

1. foreach 문

foreach 문 사용 시 주의점

- 반복 변수에는 현재 요소 값의 사본이 저장된다.
 - 참조 자료형의 경우 참조값의 사본이 저장된다.
- 반복 변수에 다른 값을 대입해도 배열이나 컬렉션에 저장된 요소 자체가 바뀌지는 않는다.
- foreach 문은 현재 요소의 index 정보를 직접 제공하지 않는다.
- 순회 순서는 컬렉션의 종류와 구현에 따라 달라진다.

2. var 키워드

var 키워드란?

- var 키워드는 지역 변수의 자료형을 컴파일러가 추론하도록 하는 문법 요소이다.
- 변수의 실제 자료형은 컴파일 시점에 결정되며 결정된 이후에는 변경되지 않는다.
- 선언과 동시에 초기화가 반드시 필요하다.

```
var 변수명 = value;
```

```
var userCount = 5;      // int로 추론됨
```

```
var userName = "Alice"; // String으로 추론됨
```


2. var 키워드

foreach 문에서 var 키워드 사용하기

- foreach 문에서 변수의 자료형을 var 키워드로 선언할 수 있다.
- 반복 변수의 실제 자료형은 순회 대상의 요소 자료형을 기준으로 컴파일 시점에 추론된다.

```
for (var 변수명 : 배열 또는 컬렉션) {  
}
```

2. var 키워드

ArrayList에서의 var 키워드 사용

```
ArrayList<String> userNames = new ArrayList<>();  
userNames.add("Alice");  
userNames.add("Bobby");  
userNames.add("Charlie");  
  
for (var userName : userNames) {  
    // userName은 String으로 추론됨  
}
```

2. var 키워드

HashSet에서의 var 키워드 사용

```
HashSet<String> processedEmails = new HashSet<>();  
processedEmails.add("alice@example.com");  
processedEmails.add("bobby@example.com");  
processedEmails.add("charlie@example.com");  
  
for (var email : processedEmails) {  
    // email은 String으로 추론됨  
}
```

2. var 키워드

HashMap에서의 var 키워드 사용

```
HashMap<String, Long> lastLoginTimesByUsername = new HashMap<>();
lastLoginTimesByUsername.put("alice", 1735689600L);
lastLoginTimesByUsername.put("bobby", 1740830400L);
lastLoginTimesByUsername.put("charlie", 1750002600L);

for (var entry : lastLoginTimesByUsername.entrySet()) {
    // entry는 Map.Entry<String, Long>으로 추론됨
}
```

2. var 키워드

코딩 표준: var 키워드

- 초기화 식만으로 변수의 자료형과 의미를 명확히 판단하기 어려운 경우에 var 키워드를 사용하면 코드의 가독성이 심각하게 저하될 수 있다.

```
// 잘못된 예
```

```
var result = process();
```

```
var data = getData();
```

- var 키워드는 다음 조건을 만족할 때만 제한적으로 사용한다.
 - 초기화 식만으로 변수의 자료형이 명확히 드러나는 경우에 var 키워드를 사용할 수 있다.

```
var userName = "Alice";
```

```
var userNames = new ArrayList<String>();
```

- foreach 문에서 순회 대상 요소의 자료형을 쉽게 알 수 있을 때 var 키워드를 사용할 수 있다.

Thank you

Wishing you an enjoyable and meaningful learning experience.
May you grow into an engineer who learns with joy and shares with others.

For inquiries, contact: potterLim0808@gmail.com

